

Solution

1. Project idea

We simulate an **e-commerce streaming system**:

```
Orders JSON files
    ↓
Spark Structured Streaming
    ↓
Deduplication
    ↓
Watermark for late events
    ↓
Join with customer static data
    ↓
Windowed aggregation
    ↓
foreachBatch
    ↓
Parquet + JSON
```

2. Input data

customers.json:

```
{ "customer_id": 101, "customer_name": "John", "city": "Toronto", "segment": "Premium" }
{ "customer_id": 102, "customer_name": "Sarah", "city": "New York", "segment": "Regular" }
{ "customer_id": 103, "customer_name": "Mike", "city": "London", "segment": "Premium" }
```

Streaming orders:

```
{ "order_id": "ORD001", "customer_id": 101, "product": "Laptop", "category": "Electronics", "quantity": 1, "price": 1200.0, "event_time": "2026-08-26T10:00:00" }
{ "order_id": "ORD002", "customer_id": 102, "product": "Phone", "category": "Electronics", "quantity": 2, "price": 700.0, "event_time": "2026-08-26T10:01:00" }
{ "order_id": "ORD003", "customer_id": 101, "product": "Mouse", "category": "Accessories", "quantity": 3, "price": 30.0, "event_time": "2026-08-26T10:02:00" }
{ "order_id": "ORD002", "customer_id": 102, "product": "Phone", "category": "Electronics", "quantity": 2, "price": 700.0, "event_time": "2026-08-26T10:01:00" }
```

Notice that ORD002 appears twice. That's intentional so we can practice **deduplication**.

3. Complete PySpark solution

```

# =====
# ADVANCED SPARK STRUCTURED STREAMING PROJECT
# Joins + Stateful Aggregation + Deduplication
# + Late Events + Watermark + foreachBatch
# =====

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# =====
# 1. CREATE SPARK SESSION
# =====

spark = (
    SparkSession.builder
        .appName("Advanced Streaming Project")
        .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

print("Spark version:", spark.version)

# =====
# 2. DEFINE ORDER SCHEMA
# =====

order_schema = StructType([
    StructField("order_id", StringType(), False),
    StructField("customer_id", IntegerType(), False),
    StructField("product", StringType(), True),
    StructField("category", StringType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("price", DoubleType(), True),
    StructField("event_time", TimestampType(), True)
])

# =====
# 3. READ CUSTOMERS AS STATIC DATA
# =====

customers = (
    spark.read
        .json("hdfs://localhost:9000/data/customers/customers.json")
)

print("Customers:")
customers.show()

# =====
# 4. READ ORDERS AS STREAM
# =====

```

```

orders = (
    spark.readStream
        .schema(order_schema)
        .json("hdfs://localhost:9000/data/orders")
)

# =====
# 5. CALCULATE TOTAL ORDER VALUE
# =====

orders = orders.withColumn(
    "total_amount",
    col("quantity") * col("price")
)

# =====
# 6. DEDUPLICATION
# =====

# Keep only one record for each order_id.
#
# Watermark allows Spark to eventually remove old state
# used for deduplication.

orders_dedup = (
    orders
        .withWatermark("event_time", "10 minutes")
        .dropDuplicates(["order_id"])
)

# =====
# 7. JOIN STREAM WITH STATIC CUSTOMER DATA
# =====

enriched_orders = (
    orders_dedup
        .join(
            customers,
            on="customer_id",
            how="left"
        )
)

# =====
# 8. HANDLE LATE EVENTS
# =====

# Spark uses event_time rather than processing time.
#
# Example:
#
# event_time = 10:00

```

```

# event_time = 10:05
# event_time = 09:55
#
# The watermark tells Spark how long to wait for late data.

enriched_orders = (
    enriched_orders
    .withWatermark("event_time", "10 minutes")
)

# =====
# 9. WINDOWED STATEFUL AGGREGATION
# =====

windowed_sales = (
    enriched_orders
    .groupBy(
        window(
            col("event_time"),
            "5 minutes"
        ),
        col("city"),
        col("category")
    )
    .agg(
        sum("total_amount").alias("total_sales"),
        sum("quantity").alias("total_quantity"),
        count("order_id").alias("order_count")
    )
)

# =====
# 10. SELECT FINAL COLUMNS
# =====

result = (
    windowed_sales
    .select(
        col("window.start").alias("window_start"),
        col("window.end").alias("window_end"),
        col("city"),
        col("category"),
        col("total_sales"),
        col("total_quantity"),
        col("order_count")
    )
)

# =====
# 11. FOREACHBATCH FUNCTION
# =====

def process_batch(batch_df, batch_id):

```

```

print("=" * 60)
print("PROCESSING BATCH:", batch_id)
print("=" * 60)

# Don't process empty batches
if batch_df.isEmpty():
    print("Empty batch")
    return

# Show current batch
batch_df.show(truncate=False)

# -----
# WRITE PARQUET
# -----

(
    batch_df
    .write
    .mode("append")
    .parquet(
        "hdfs://localhost:9000/data/output/parquet"
    )
)

# -----
# WRITE JSON
# -----

(
    batch_df
    .write
    .mode("append")
    .json(
        "hdfs://localhost:9000/data/output/json"
    )
)

print("Batch written successfully.")

# =====
# 12. START STREAMING QUERY
# =====

query = (
    result
    .writeStream
    .foreachBatch(process_batch)
    .outputMode("update")
    .option(
        "checkpointLocation",
        "hdfs://localhost:9000/data/output/checkpoint"
    )
    .trigger(processingTime="10 seconds")
    .start()
)

```

```
# =====  
# 13. WAIT FOR STREAMING  
# =====
```

```
query.awaitTermination()
```

What each advanced feature is doing

Deduplication

```
.dropDuplicates(["order_id"])
```

If Spark receives:

```
ORD001  
ORD002  
ORD002  
ORD003
```

the result becomes:

```
ORD001  
ORD002  
ORD003
```

The watermark limits how long Spark needs to maintain deduplication state.

Late events

```
.withWatermark("event_time", "10 minutes")
```

Suppose Spark has received:

```
10:00  
10:05  
10:08
```

and then receives an event whose event time is:

```
10:02
```

Spark can still process it because it is within the allowed lateness.

The important concept is:

Event time
↓
Watermark
↓
How much late data Spark will tolerate

Static join

```
orders_dedup.join(  
    customers,  
    on="customer_id",  
    how="left"  
)
```

An order:

```
customer_id = 101
```

gets customer information:

```
customer_id = 101  
customer_name = John  
city = Toronto  
segment = Premium
```

So the streaming order becomes enriched with customer information.

Stateful aggregation

This:

```
.groupBy(  
    window("event_time", "5 minutes"),  
    "city",  
    "category"  
)  
.agg(  
    sum("total_amount"),  
    sum("quantity"),  
    count("order_id")  
)
```

requires Spark to maintain aggregation state across micro-batches.

For example:

```
10:00 - 10:05  
Toronto
```

Electronics

Spark maintains something similar to:

```
sales = 1900
quantity = 3
orders = 2
```

Then new events can update that state.

foreachBatch

This is the important part:

```
def process_batch(batch_df, batch_id):
```

Spark gives you every micro-batch as a **normal DataFrame**.

That means you can perform batch operations such as:

```
batch_df.write.parquet(...)
batch_df.write.json(...)
```

You can even write the same batch to multiple destinations.

```
Streaming  ──> foreachBatch
                ┌──> Parquet
                └──> JSON
```

Important correction

This example uses **stream-to-static join**, which is the easier join type.

For the truly advanced version, the next step would be:

```
Orders stream
      +
Payments stream
      ↓
Stream-to-stream join
      ↓
Watermark on BOTH streams
      ↓
Time-range join
      ↓
State management
      ↓
foreachBatch
```

That is the version I'd consider the **real**  **advanced challenge**.